

Talk

A phonetic encoding with one token per effective sound,
for speech and language models

Lance Pollard

`lancejpollard@gmail.com`

July 2026

Abstract

Speech and language models need a written form for pronunciation that a tokenizer can handle well. The International Phonetic Alphabet is accurate and well suited to its own purpose, but less than ideal in this setting. A single sound is often a base letter followed by several combining marks, some symbols sit in the supplementary planes of Unicode, and a subword or byte tokenizer splits one pronunciation across several tokens with no fixed relation to the sounds. Talk is a phonetic encoding built for this setting. It writes every sound as a base consonant or vowel plus a small set of modifiers, in plain ASCII that a normal keyboard can type. It also maps each canonical sound to a single code point, so a whole word becomes one token per sound, using the Hangul syllable block for individual sounds and Chinese characters for consonant and vowel clusters. The map from sound to code point is fixed and append-only, so a model built on it keeps working as the inventory grows. Talk converts to and from IPA, and it splits words into syllables. It has no runtime dependencies and ships for Python and TypeScript. This paper describes the encoding, the token form, and how to use it. The system is implemented and in use, and coverage of the remaining IPA symbols is being completed.

1. Introduction

A model that reasons about how words sound needs a written form for sound. The common choice is the International Phonetic Alphabet, which is accurate, widely used among linguists, and holds a large amount of recorded pronunciation data. For a model pipeline it raises a few practical problems.

A single sound is often a base letter followed by one or more combining diacritics, so the number of characters does not match the number of sounds. Some of the newer symbols sit in the supplementary planes of Unicode, which many tokenizers handle as several units. And the subword or byte tokenizers that most models use break one pronunciation into several tokens that have no fixed relation to the phones. A model then has to learn that a scattered set of fragments stands for one sound.

Talk is a phonetic encoding designed around these problems. It has two written forms. The first is a plain ASCII spelling, where each sound is a base consonant or vowel followed by modifiers, so a person can type pronunciations on a normal keyboard and a program can read them without special handling. The second is

a machine form, where each canonical sound is a single code point, so a word becomes a compact string of one token per sound. A short example, in Python:

```
import talk

# prajna, written in IPA
talk.talk_to_ipa("pradjy~ny~a_") # => "pradzɲa:"

# one Hangul code point per sound
talk.machine("pradjy~ny~a_") # => a 7 code point string

# the sounds, each with its features
talk.segment("pradjy~ny~a_") # => [p, r, a, d, jy~, ny~, a_]

# and the reverse, IPA back to talk
talk.ipa_to_talk("predzɲa:") # => "pradjy~ny~a_"
```

The contribution is a small set of design choices that work together.

- A phonetic alphabet in plain ASCII, where each sound is a base plus modifiers in a fixed order, so every sound has exactly one spelling.
- A machine form that maps each canonical sound to one code point, so a word is one token per sound. The Hangul syllable block holds the individual sounds, and Chinese characters hold consonant and vowel clusters.
- A fixed, append-only map from sound to code point, so a token never changes meaning across releases.
- Conversion to and from IPA, and splitting of a word into syllables.
- Implementations for Python and TypeScript with no runtime dependencies.

Talk sits alongside earlier work. X-SAMPA gives an ASCII spelling of IPA for keyboard entry [2], and tools such as Epitran map orthography to IPA for many languages [1]. Subword methods such as byte-pair encoding and unigram models are how model vocabularies are usually built [4, 5]. Talk aims instead at a phone-level and cluster-level vocabulary directly, with a fixed one-token-per-sound map and a bridge to IPA in both directions.

The intended readers build text-to-speech, speech recognition, grapheme-to-phoneme, or multilingual systems, and want a compact and stable phonetic vocabulary in place of raw IPA. The source is available [3].

2. The encoding

Talk describes a sound as a base plus modifiers. The bases are the consonants and vowels of the IPA charts, written with Latin letters and a few case and punctuation conventions. Uppercase marks a variant of a letter, so **t** and **T** are two related places of articulation, and **a** and **A** are two related vowels. A base on its own is a common sound. A base with modifiers is a more specific one.

share should grow. Real pronunciation varies within a single word and across speakers, so the finest IPA distinctions, such as small differences in tongue-root position or degree of rounding, are often below what a discrete model can use or a listener can reliably hear. Talk folds those into the nearest supported sound. When the last five percent matters, IPA is still there, and talk converts to it.

Conversion runs in both directions. `talk_to_ipa` reads a talk string and returns IPA. `ipa_to_talk` reads IPA and returns talk. The mapping is approximate on the IPA side. Several fine IPA symbols map to the same talk sound, and a few marks, such as the affricate tie bar and the relative-articulation diacritics, are not represented. This is the ninety-five percent choice made concrete, and those marks are candidates to add later.

3. The machine form

The set of canonical sounds is finite. Counting every base with every valid combination of modifiers gives about two thousand sounds. Because the set is finite and fixed, each sound can be given its own code point, and a word can be written as one code point per sound.

Talk uses the Hangul syllable block for this. That block is a large contiguous run of code points, more than eleven thousand of them, each a single code point rather than a base plus combining marks. Assigning one Hangul code point to each talk sound gives a model a finite and normalized phone vocabulary, where a pronunciation is a short sequence of whole tokens and never splits across fragments. The choice of Hangul is a matter of fit. It is a large, contiguous run of code points, rendered by common fonts, and its characters share one fixed width. A token string then reads as a clear, evenly spaced sequence that is easy to count and align, which suits a discrete phone vocabulary. A sound like `mh!im`, a voiceless `m` then `i` then `m`, becomes a three code point string, one token per sound.

A base and all of its modifiers collapse onto one token. This is what one token per effective sound means. The examples below stack several features onto a single sound, and each is still one Hangul code point.

IPA	Talk	Token	Features
t	t	것	base consonant
t ^h	th~	갇	aspirated
t'	t!	경	ejective
k ^{hw}	kw~h~	꺄	aspirated, labialized
q ^w	K!w~	꺆	ejective, labialized
a	a	덧	base vowel
a:	a_	델	long vowel
a ¹	a+	덜	high tone
ã ¹	a&+	뎡	nasalized, high tone

The map from sound to code point is fixed and append-only. A code point is never reassigned once given, and a new sound takes the next free slot. A run that adds nothing leaves the map unchanged. This matters for anyone who trains on the tokens. A model, an index, or a dataset built on the map keeps its meaning after an upgrade, since old tokens still point to the same sounds.

There is a second granularity above the single sound. Talk splits a word into syllables, and a syllable is built from clusters, an onset consonant cluster, a vowel cluster for the nucleus, and a coda consonant cluster. Each cluster can be tokenized too, and for this talk uses Chinese characters, tens of thousands of single code points. A consonant cluster such as `skr` or a vowel cluster gets one Chinese character, so a syllable becomes a short run of cluster tokens rather than a longer run of phone tokens. This is useful where a shorter sequence helps, since a cluster sequence is coarser than a phone sequence while still tracking how the word is put together. Chinese characters are used here for the same reasons, a large set with consistent fixed-width glyphs and common font coverage. The Chinese set is far larger than the Hangul block, which matches the expectation that clusters outnumber single sounds, since clusters are combinatorial while the sound inventory is fixed. The particular code points are an implementation choice. The property that carries is one token per unit, whether the unit is a sound or a cluster.

IPA	Talk	Token
skr	skr	屎
spl	spl	穴
spr	spr	帆
str	str	吁
ltj	ltx	漲
bw	bw	幘
nd	nd	憫
g	g	摑
ai	ai	鼻
au	au	幘
ei	ei	痧
a	a	擘

The cluster inventory is where the system is still growing. The current set covers a subset of consonant and vowel clusters and ships in the TypeScript library, with the Python port next. A richer inventory, drawn from large IPA word lists gathered from the web, is the next step, so the syllable side widens over time. The sound tokens beneath it stay fixed as it grows.

4. Use cases

Talk ships as a library for Python and TypeScript, with no runtime dependencies, and the functions are the same in both. The introduction shows a short Python example. The TypeScript package is `@cluesurf/talk`.

A few potential use cases follow for inspiration. There are of course many other applications as well.

A text-to-speech or grapheme-to-phoneme front end turns text or IPA into talk, then feeds the acoustic model one token per sound. The model sees a normalized phone vocabulary of fixed size, in place of raw IPA that a byte tokenizer would fragment. The same front end can emit cluster tokens where the downstream model works at the syllable level.

A speech recognition or forced alignment system that labels audio at the phone or cluster level can use talk tokens as its label set. Because the map is fixed, a label set built once stays valid as the inventory grows.

A multilingual model can share one phonetic vocabulary across languages, since talk writes every language in the same base and modifier system. A word in one language and a similar sounding word in another land on overlapping tokens, which grapheme based vocabularies do not give.

A language learning tool can show the ASCII form, which reads much like the pronunciation guides that dictionaries already use, together with the syllable split. A learner then gets a readable spelling and clear syllable breaks without needing to read IPA.

5. Scope and status

The encoding and both libraries are implemented and in use. The Python and TypeScript packages convert between IPA and talk, produce the machine tokens, and expose the sound inventory, with no runtime dependencies.

The aim is to cover the full IPA inventory in the ASCII form and in the machine tokens. Most of it is covered now. A small number of rarer symbols are still being added, and because the sound-to-token map grows by appending, this happens without disturbing existing tokens.

The syllable splitter is available in the TypeScript library and handles the syllable shapes of common languages. Some rarer material, such as clicks, is not split yet, and the Python port of the splitter is in progress. The syllable layer sits above the sound encoding, so the phone tokens are unaffected by its coverage.

The design is settled and the remaining work is additive. New sounds extend the inventory, new rules extend the splitter, and neither changes the meaning of what is already there.

6. Conclusion

Talk gives a phonetic encoding with two forms that serve different needs. The ASCII form is typeable and readable, and stays close to how pronunciation guides are already written. The machine form is one code point per sound, on a fixed and append-only map, so a word becomes a compact and steady sequence of tokens. Both forms convert to and from IPA, and a word can be split into syllables and clusters for the tasks that want a coarser unit.

The goal is a phonetic vocabulary that a speech or language model can use directly, one that is finite, normalized, and steady across releases, with IPA available whenever the finest detail is needed. The libraries are ready to try, and the source is open [3].

References

- [1] David R. Mortensen, Siddharth Dalmia, and Patrick Littell. Epitran: Precision G2P for many languages. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC)*, 2018.
- [2] John C. Wells. Computer-coding the IPA: a proposed extension of SAMPA. *University College London*, 1995.
- [3] Lance Pollard. Talk: a phonetic encoding. <https://github.com/cluesurf/talk>, 2026. Software repository.
- [4] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016.
- [5] Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, 2018.

A. Supported IPA Consonants and Vowels

The consonant symbols talk currently encodes.

[illegible]

The vowel symbols.

i y i ɥ u ɪ ʏ ʊ e ø ə ɐ o ɘ ɵ
 ə ʎ ɔ ɛ œ ɜ ɞ ʌ ɔ æ ɐ a œ ä ɑ ɒ